# React Front-End Project Structure Documentation

This document describes a scalable and maintainable folder structure for React front-end projects. It is suitable for small, medium, and large applications such as dashboards, admin panels, and general web apps.

---

## 📁 Project Structure Overview

```
src/
│
├── assets/
│   ├── images/
│   ├── icons/
│   └── styles/
│       ├── _variables.scss
│       ├── _mixins.scss
│       └── main.scss
│
├── components/
│   ├── common/
│   │   ├── Button/
│   │   │   ├── Button.jsx
│   │   │   └── Button.module.scss
│   │   └── Loader/
│   └── layout/
│       ├── Navbar/
│       ├── Footer/
│       └── Sidebar/
│
├── pages/
│   ├── Home/
│   │   ├── Home.jsx
│   │   └── Home.module.scss
│   ├── Login/
│   └── NotFound/
│
├── services/
│   ├── api.js
│   ├── auth.service.js
│   └── user.service.js
│
├── hooks/
```

```
│   ├── useAuth.js
│   └── useDebounce.js
│
├── context/
│   ├── AuthContext.jsx
│   └── ThemeContext.jsx
│
├── store/
│   ├── index.js
│   └── slices/
│
├── routes/
│   └── AppRoutes.jsx
│
├── utils/
│   ├── constants.js
│   ├── helpers.js
│   └── validators.js
│
├── App.jsx
├── main.jsx
└── index.css
```

---

## 📂 Folder Descriptions

### ✂️ assets

Contains all static files used across the application.

**Includes:** - Images (PNG, JPG, SVG) - Icons - Global styles (CSS / SCSS)

---

### ✒️ components

Reusable UI components that can be shared across multiple pages.

`components/common`

Generic components such as: - Buttons - Loaders - Modals - Inputs

Each component should have its own folder.

`components/layout`

Layout-related components used to structure the application: - Navbar - Sidebar - Footer

---

### 🛏️ pages

Represents application pages. Each page usually corresponds to a route.

**Examples:** - Home - Login - Dashboard - NotFound

Each page has its own folder and styling file.

---

### 🌂 services

Handles all API calls and external data communication.

**Best practice:** - Do not call APIs directly inside components or pages - Centralize API logic here

**Examples:** - `api.js` – Axios configuration - `auth.service.js` – Authentication APIs - `user.service.js` – User-related APIs

---

### 🌡️ hooks

Custom React hooks to encapsulate reusable logic.

**Examples:** - `useAuth` - `useFetch` - `useDebounce`

---

### 🎒 context

Used when working with React Context API for global state management.

**Examples:** - Authentication context - Theme context - Language context

---

### 🎓 store

Used if you are using a state management library such as: - Redux Toolkit - Zustand

**Contains:** - Store configuration - Reducers / slices

---

### 🎩 routes

Centralized routing configuration for the application.

**Example:** - `AppRoutes.jsx` defines all application routes in one place

---

### 🎽 utils

Utility functions and shared helpers.

**Includes:** - Constants - Validation functions - Helper methods

---

## 🔗 Best Practices

- Keep components small and reusable
- Avoid heavy logic inside JSX
- Separate UI, business logic, and data access
- Each component should have its own folder
- Centralize API calls in `services`
- Use hooks for shared logic

---

## 📌 Notes

This structure is flexible and can be extended to support: - Dashboards - Multi-language applications - Role-based access control - Server-side rendering (Next.js)

---